

Lecture review

Exercise 1

Consider a queue implemented using a linked list. The queue supports the standard operations **enqueue** and **dequeue**.

Write a function **moveNthFront** that takes as a parameter a positive integer n and moves the n^{th} element of the queue to the front. The order of the remaining elements must remain unchanged.

For example, suppose:

queue = {5, 11, 34, 67, 43, 55} and $n = 3$.

After calling **moveNthFront**, the queue becomes:

queue = {34, 5, 11, 67, 43, 55}.

Implement the following member functions for a linked-list based **Queue** class:

- `void enqueue(int value);`
- `int dequeue();`
- `void moveNthFront(int n);`

Assume the queue is not empty and that $1 \leq n \leq$ (number of elements in the queue).

Exercise 2

Consider a queue implemented using a singly linked list with pointers **front** and **rear**. In addition to the standard queue operations **enqueue** and **dequeue**, implement a function **moveNthToRear** that takes a positive integer n as a parameter.

The function moves the n^{th} element (counting from the front, starting at 1) to the rear of the queue. The relative order of all other elements must remain unchanged. The operation must be performed by rearranging pointers only (no creation or deletion of nodes).

For example, suppose:

queue = {10, 20, 30, 40, 50, 60} and $n = 2$.

After calling **moveNthToRear**, the queue becomes:

queue = {10, 30, 40, 50, 60, 20}.

Additional Requirements:

- The implementation must run in $O(n)$ time.
- No additional data structures (arrays, stacks, or extra linked lists) may be used.
- Do not modify the data stored in the nodes.

Implement the following member functions for the linked-list based **Queue** class:

- `void enqueue(int value);`
- `int dequeue();`
- `void moveNthToRear(int n);`

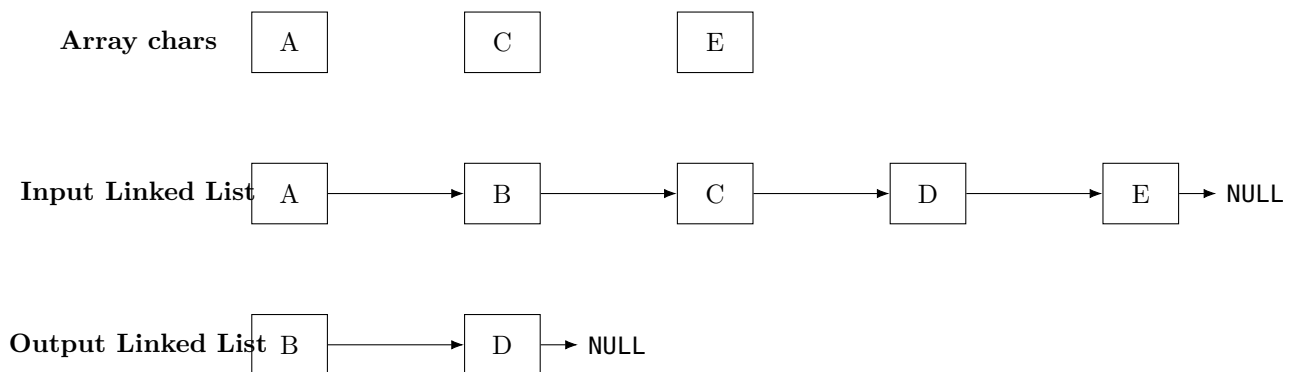
Assume the queue is not empty and that $1 \leq n \leq$ (number of elements in the queue).

Exercise 3

You are given an array of characters **chars** and the head of a linked list whose nodes store character values.

Return the head of the modified linked list after removing all nodes whose values appear in the array **chars**.

Visualization of Linked List Modification



Exercise 4

You are given a stream of stock price updates for a single company. Each update consists of:

- A **timestamp** (an integer), and
- The **price** of the stock at that timestamp.

However, the updates have two complications:

1. The timestamps may arrive **out of order**.
2. A timestamp may appear more than once. If this happens, the new price **replaces** (corrects) the previous price for that timestamp.

Your task is to design a data structure that efficiently supports price updates and queries about the stocks performance.

Required Operations

Design and implement a class called **StockPrice** that supports the following operations:

- **StockPrice()**
Initializes the object. Initially, there are no price records.

- `void update(int timestamp, int price)`
Records or updates the price of the stock at the given timestamp. If the timestamp already exists, replace the old price with the new one.
- `int current()`
Returns the price at the **latest timestamp** recorded so far.
- `int maximum()`
Returns the **highest price** among all current records.
- `int minimum()`
Returns the **lowest price** among all current records.

Example

Input

```
["StockPrice", "update", "update", "current", "maximum",  
 "update", "maximum", "update", "minimum"]
```

```
[[], [1, 10], [2, 5], [], [], [1, 3], [], [4, 2], []]
```

Output

```
[null, null, null, 5, 10, null, 5, null, 2]
```

Explanation

```
StockPrice stockPrice = new StockPrice();
```

```
stockPrice.update(1, 10);  
// Records: {1 -> 10}
```

```
stockPrice.update(2, 5);  
// Records: {1 -> 10, 2 -> 5}
```

```
stockPrice.current();  
// Latest timestamp is 2 -> returns 5
```

```
stockPrice.maximum();  
// Maximum price is 10 -> returns 10
```

```
stockPrice.update(1, 3);  
// Corrects timestamp 1 from 10 to 3  
// Records: {1 -> 3, 2 -> 5}
```

```
stockPrice.maximum();
// Maximum price is now 5 -> returns 5
```

```
stockPrice.update(4, 2);
// Records: {1 -> 3, 2 -> 5, 4 -> 2}
```

```
stockPrice.minimum();
// Minimum price is 2 -> returns 2
```

Constraints

- $1 \leq \text{timestamp}, \text{price} \leq 10^9$
- At most 10^5 total calls will be made to `update`, `current`, `maximum`, and `minimum`.
- The methods `current`, `maximum`, and `minimum` will only be called after at least one `update`.

Exercise 5

Imagine you are designing a parking management system for a small coffee shop. The shop has a **circular drive-thru lane** with k parking spots arranged in a circle.

Cars enter one by one and leave in the same order they arrived. This follows the **FIFO (First In First Out)** principle.

Unlike a straight parking lane, once a car leaves, that spot becomes reusable even if it is at the beginning. The last parking spot connects back to the first, forming a **ring structure**.

Requirements

Design a class called `CircularDriveThru` that supports:

- **`CircularDriveThru(k)`**: Initialize the system with k parking spots.
- **`parkCar(carNumber)`**: Insert a car into the next available spot.
- **`carLeaves()`**: Remove the car that arrived first.
- **`frontCar()`**: Return the first car (or -1 if empty).
- **`lastCar()`**: Return the most recently parked car (or -1 if empty).
- **`isEmpty()`**: Check whether the parking lane is empty.
- **`isFull()`**: Check whether the parking lane is full.

Example

```
CircularDriveThru lane = new CircularDriveThru(3);
```

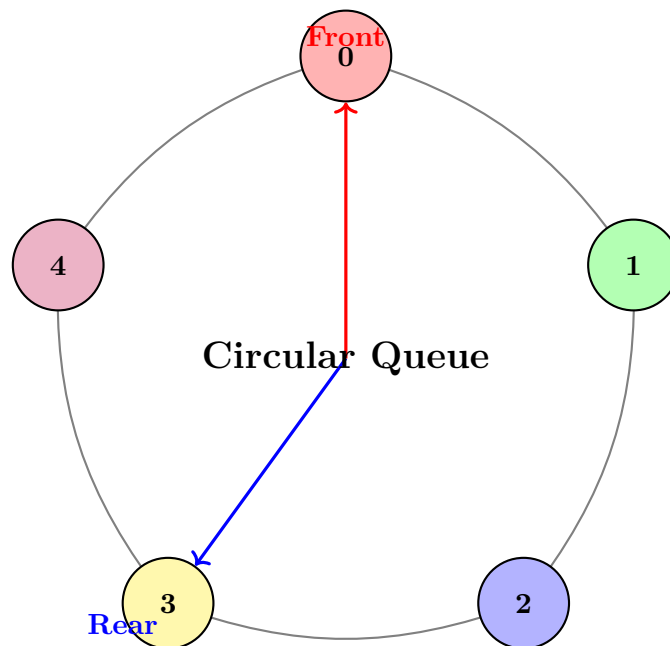
```
lane.parkCar(101); % true
lane.parkCar(102); % true
lane.parkCar(103); % true
```

```
lane.parkCar(104); % false (full)

lane.carLeaves(); % 101 leaves
lane.parkCar(104); % true (spot reused)
```

Below is a visual representation of a circular parking system with 5 spots.

Below is a visual representation of a circular parking system with 5 spots.



- The parking spots are arranged in a circular fashion.
- The **front pointer** indicates which car will leave next.
- The **rear pointer** indicates where the next car will park.
- When the rear reaches the last spot, it wraps around to spot 0:

$$\text{rear} = (\text{rear} + 1) \bmod k$$

- Similarly, when a car leaves:

$$\text{front} = (\text{front} + 1) \bmod k$$

This ensures efficient reuse of parking spaces and avoids wasted space, which would occur in a linear queue.

Exercise 6

Define a class called **Stack** that implements a stack using a dynamic array. The class should have the following public methods:

Method	Description
<code>Stack(int capacity)</code>	constructs a stack with an initial capacity of capacity . If capacity is not provided, default to 10.
<code>void push(int value)</code>	adds an element to the top of the stack. If the stack is full, it should **double** the capacity of the array before adding the element.
<code>int pop()</code>	removes and returns the top element of the stack. If the stack is empty, throw an exception.
<code>int peek()</code>	returns the top element of the stack **without removing** it. If the stack is empty, throw an exception.
<code>bool isEmpty()</code>	returns true if the stack has no elements, otherwise returns false .
<code>int getSize()</code>	returns the number of elements currently stored in the stack.
<code>void reverseMiddleHalf()</code>	reverses the **middle half** of the stack while keeping the top and bottom quarters unchanged. If the stack has an odd number of elements, the middle element remains in its position.
<code>string toString()</code>	returns a string representation of the stack in the format: <code>[elem1, elem2, ..., elemN]</code> where elemN is the top of the stack.

Examples:

```
Stack s(10);
s.push(10);
s.push(20);
s.push(30);
s.push(40);
s.push(50);
s.push(60);
s.push(70);
s.push(80);
s.push(90);
s.push(100);
cout << s.toString();
```

Before calling `reverseMiddleHalf()`:

```
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

After calling `reverseMiddleHalf()`:

[10, 20, 70, 60, 50, 40, 30, 80, 90, 100]

Also make `Stack` objects comparable to each other by overloading the `<` operator. A `Stack` with more elements should be considered "less" than a stack with fewer elements so that they appear earlier in a sorted list. If two stacks have the same number of elements, they should be compared by the sum of their elements, with stacks having a larger sum being "less". If stacks are still equal, compare their top element, with the stack having a smaller top element being considered "less".

Example:

Given stacks:

Stack A: [10, 20, 30], Stack B: [5, 15], Stack C: [5, 10, 20], Stack D: [10, 25] ,
Stack E: [10, 20, 30]

Sorting order explanation:

1. **Stack A vs. Stack B** - A has **3 elements**, B has **2 elements** $\Rightarrow$ **A appears first** ($A < B$ is true).
2. **Stack A vs. Stack C** - Both have **3 elements** $\Rightarrow$ Compare sums: - A: $10 + 20 + 30 = 60$ - C: $5 + 10 + 20 = 35$ - Since A has a **larger sum**, A appears first ($A < C$ is true).
3. **Stack C vs. Stack D** - C has **3 elements**, D has **2 elements** $\Rightarrow$ **C appears first** ($C < D$ is true).
4. **Stack A vs. Stack E** - Both have **same elements**, so they are considered **equal** in ordering.

Final sorted order:

[Stack A, Stack E, Stack C, Stack D, Stack B]

Exercise 7

You are designing software for the **emergency room (ER)** of a hospital. Patients arrive continuously, and each patient has:

- **Name**
- **Arrival time**
- **Severity level** (1–10, where 10 is most critical)

The ER doctor wants to treat patients in the **order of severity**, not just the order of arrival. If two patients have the same severity, the one who **arrived earlier** should be treated first.

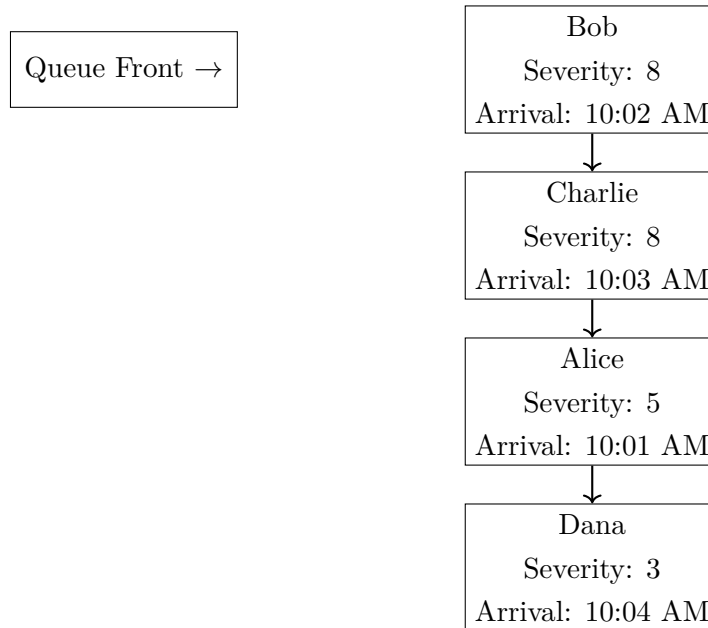
Your task is to design a system that supports:

1. Add a new patient to the queue.
2. Treat the next patient (remove the highest priority patient from the queue).
3. Peek at the next patient to be treated without removing them.

Example Patient Data

Name	Severity	Arrival Time
Alice	5	10:01 AM
Bob	8	10:02 AM
Charlie	8	10:03 AM
Dana	3	10:04 AM

Treatment order using a priority queue: Bob, Charlie, Alice, Dana



Exercise 8

You are designing software for an **airport control tower** to manage airplanes landing and taking off. The system handles two types of operations:

1. **Normal Takeoff Queue:** Planes waiting to take off are handled in **last-in, first-out (LIFO)** order. This requires a **stack**.
2. **Emergency Landing Queue:** Planes requesting emergency landings are handled in **priority order** based on emergency severity or low fuel. This requires a **priority queue**.

Each plane has:

- **Flight number**
- **Type of operation** (Takeoff / Landing)
- **Priority/severity** (only for landing emergencies)
- **Arrival time in the queue/stack**

Operations

1. Push a plane into the takeoff stack if its a normal takeoff.

2. Add a plane into the emergency landing priority queue if it requests emergency landing.
3. Next action:
 - If the landing queue is non-empty, land the plane with the highest priority.
 - Otherwise, allow the top plane from the takeoff stack to take off.

Example

Takeoff Stack (LIFO)

Flight	Arrival Time
AB123	10:00 AM
CD456	10:05 AM
EF789	10:10 AM

Emergency Landing Priority Queue

Flight	Severity	Arrival Time
GH101	9	10:02 AM
IJ202	10	10:06 AM
KL303	8	10:09 AM

Next actions: Land IJ202, Land GH101, Take off EF789

Stack and Queue example

